

SMC Trading Eye / Trading Brain Architecture Refactor

你现在需要对当前 `smc_test` 项目进行一次架构级审查与渐进式重构。

这不是要求从零重新实现一套 SMC 系统，而是在充分理解当前代码库已有实现的前提下，重新定义并落实：

- * Trading Eye / 交易眼睛
 - * Trading Brain / 交易大脑
 - * Execution / 执行器
 - * SMC Semantic Specification / 预注册语义体系
 - * Signal Research
 - * Execution Research
 - * OHLCV 与 MBO 验证体系
 - * 多时间框架状态模型
 - * 可回放、可审计、无未来泄漏的研究与实盘架构
-

一、任务开始前的强制要求

在修改任何代码之前：

1. 完整检查当前 `smc_test` 代码库。
2. 明确当前已经存在：
 - * 哪些数据处理能力；
 - * 哪些 Swing / BOS / MSS / FVG / OB / Liquidity 等 SMC 语义；
 - * 哪些多时间框架逻辑；
 - * 哪些状态对象；
 - * 哪些研究模块；
 - * 哪些 execution / entry / fill / risk 模块；
 - * 哪些测试；
 - * 哪些配置；
 - * 哪些已有实现可以复用。
3. 建立：
 - * Existing Capability Map
 - * Gap Analysis
 - * Refactor Map
4. 不要因为新的目录结构更整齐，就重新实现已经正确工作的功能。
5. 优先：

- * 复用；
- * 提取；
- * 重命名；
- * 拆分职责；
- * 增加 schema；
- * 增加 replay；
- * 增加测试；

而不是大规模推倒重写。

6. 如果当前实现与目标架构存在冲突，必须说明：

- * 当前行为；
- * 问题；
- * 目标行为；
- * 最小修改方案。

7. 不要过度建设基础设施。

8. 优先保证核心交易语义、状态逻辑、research pipeline 和 execution 能正确工作。

二、最终架构定义

2.1 Trading Eye

重新定义：

Trading Eye 是一个可回放、可审计、确定性的多时间框架市场状态引擎，而不是单纯的数据观察器。

Trading Eye =

****Data Normalization**

- * Semantic Event Detection
- * Per-Timeframe State Reduction
- * Cross-Timeframe Relationship Resolution
- * Session Context
- * Market Fact Publishing******

Trading Eye 的职责是：

- * 看见市场事实；

- * 识别预注册语义；
- * 维护各周期状态；
- * 解析父子周期确定性关系；
- * 输出市场事件；
- * 输出当前市场快照。

Trading Eye 不应该：

- * 直接判断唯一 DOL；
- * 给交易概率；
- * 决定做多做空；
- * 进行最终 setup filtering；
- * 决定是否交易；
- * 管理订单。

2.2 Trading Brain

重新定义：

Trading Brain 不重复识别结构，而是维护多个可证伪的市场路径假设，根据新证据持续更新相对可信度，并最终判断是否存在可交易优势。

Trading Brain =

****DOL Hypotheses**

- * Market Path Hypotheses
- * Belief Updating
- * Signal Evaluation
- * Trade Decision******

Brain 负责：

- * 建立多个竞争性路径；
- * 维护 hypothesis；
- * 根据新事件更新 belief；
- * 对 DOL candidate 排名；
- * 判断当前 signal 是否具有统计优势；

- * 决定是否交易；
- * 产生 Trade Intent。

2.3 Execution

Execution 独立于 Eye 和 Brain：

****Entry Method Selection**

- * Order Placement
- * Cancel / Replace
- * Stop Loss
- * Take Profit
- * Position Management
- * Fill Management******

Execution 不重新解释市场结构。

三、不要使用单一状态枚举表示市场

不要创建：

```
MARKET_STATE =  
BULLISH_RETRACEMENT_DISCOUNT_WITH_FVG...
```

因为多个维度组合后会产生巨大的笛卡尔积。

使用：

Event-sourced Hierarchical State Model

即：

Immutable Events → Deterministic Reducers → Hierarchical State Objects

例如：

timeframe: 5m
structure:
 external_direction: bullish
 internal_direction: bearish
 protected_low: 21482.25
 protected_high: null
 last_bos: 21534.50
 last_mss: 21502.00
delivery:
 phase: retracement
 active_leg_direction: bearish
 displacement_score: 0.42
range:
 low: 21482.25
 high: 21541.75
 normalized_location: 0.31
 location_label: discount
liquidity:
 unswept_bsl:
 – 21541.75
 – 21556.00
 unswept_ssl:
 – 21482.25
 recently_swept:
 – 21495.50
zones:
 active_fvg:
 – lower: 21499.00
 upper: 21503.25
 active_ob:
 – lower: 21491.50
 upper: 21497.75
quality:
 data_complete: true
 semantic_version: smc_semantics_v1.0
 known_at: 2026-08-19T10:35:00-04:00

可以进一步生成方便人类理解的综合标签：

5m bearish internal retracement
inside 1H bullish delivery
1H protected low intact
5m currently in discount

但：

综合标签只是 state object 的派生输出，不能成为真实状态本身。

四、Event Sourcing 与时间语义

这是此次重构最重要的底层原则之一。

所有语义必须区分：

event_time
known_at

例如 Swing High 实际发生：

10:00

但需要右侧两根 5m K 线确认：

event_time = 10:00
known_at = 10:10

因此：

回测、研究和 replay 中，该 Swing 只能从 10:10 开始被其他逻辑使用。

禁止任何模块根据 event_time 提前使用尚未被确认的信息。

每一个 semantic event 至少必须拥有：

event_id:
semantic_type:
direction:

timeframe:
event_time:
known_at:
price:
zone:
source_event_ids:
evidence:
semantic_version:

定义：

- * event_time：市场事件实际发生时间；
- * known_at：系统第一次合法知道该事件的时间；
- * source_event_ids：该高级事件由哪些原子事件组成；
- * semantic_version：产生该事件使用的语义规范版本。

必须针对未来泄漏建立专门测试。

五、SMC Semantic Specification v1.0

第一版正式语义体系命名：

SMC Semantic Specification v1.0

代码中推荐：

smc_semantics_v1.0

后续任何语义变化必须升级版本，例如：

smc_semantics_v1.1
smc_semantics_v2.0

不能静默修改定义。

六、所有语义必须遵循五项预注册结构

每一个正式 SMC 概念必须记录：

1. Domain Meaning

它理论上试图表达什么市场现象。

2. Operational Definition

程序如何唯一、确定性地识别。

3. Hypothesized Relationship

理论认为它与哪些未来市场事件有关。

4. Falsification Criteria

什么研究结果意味着该语义本身没有价值。

5. Out-of-Sample Acceptance Criteria

满足什么条件后，该概念才能进入最终交易模型。

不能只定义“程序怎么检测”。

必须让每个 SMC 概念都可以被独立证伪。

——

七、所有参数必须预注册并冻结

例如：

k_left
k_right
prominence_ATR
break_buffer
sweep_window
acceptance_bars
min_fvg_ticks

参数只能：

Train



Validation



Freeze



Final Out-of-Sample Test

禁止根据最终 OOS 结果继续修改这些参数。

参数研究应优先寻找：

robust plateau

而不是：

single best parameter

——

八、预注册核心语义

8.1 Confirmed Swing

Domain Meaning

局部价格转折点，是整个结构树的原子。

Operational Definition

Swing High：

- * 高于左侧 kL 根；
- * 高于右侧 kR 根；
- * $\text{prominence} \geq p \times \text{ATR}$ ；
- * 只有右侧确认完成后才能使用。

Swing Low 对称。

Hypothesis

Swing prominence 越高：

- * 后续反应是否越明显；
- * 被作为 liquidity target 的概率是否更高；
- * 对后续结构定义是否更有信息量。

Falsification

如果：

- * 参数轻微变化就产生完全不同结构；
- * high-prominence swing 与普通局部高低点没有差异；

则该语义需要被否定或重新定义。

OOS Acceptance

要求：

- * deterministic replay；
- * 不同 regime 下方向一致；
- * 参数附近存在稳定平台；
- * 不是单一参数尖峰。

Swing Hierarchy

Swing 不应该仅按照 timeframe 分层。

应该通过：

- * prominence；
- * duration；
- * nesting；

获得：

micro swing
internal swing
structural swing
external swing

8.2 Structural Leg

Domain Meaning

两个相反 confirmed swings 之间的完整价格运动。

Operational Definition

从一个 confirmed swing 到下一个反向 confirmed swing。

至少记录：

- * direction;
- * amplitude;
- * duration;
- * retracement;
- * efficiency。

例如：

$$\text{Efficiency} = \frac{|\text{Close}_{\text{end}} - \text{Close}_{\text{start}}|}{\sum |\Delta \text{Close}|}$$

Efficiency 越接近 1，路径越单边。

Hypothesis

高效率 leg 是否更容易产生：

- * displacement;
- * BOS;

* continuation。

Falsification

Leg efficiency、amplitude 等变量无法稳定解释未来结构结果。

OOS Acceptance

Leg 特征必须对未来结构事件提供稳定 incremental information。

8.3 Candidate Liquidity Level

Domain Meaning

可能聚集订单、吸引价格或成为 future target 的可观察价格位置。

Operational Definition

可由预定义来源创建，例如：

- * confirmed swing;
- * equal highs / lows;
- * previous day high / low;
- * session high / low;
- * active range boundary。

命名要求

Trading Eye 必须称之为：

candidate_liquidity_level

不能在 semantic layer 中直接宣称：

institutional stops

smart money liquidity

因为这是不可直接观察的因果解释。

Hypothesis

不同类型水平的：

- * touch;
- * penetration;
- * rejection;
- * acceptance;

概率是否存在稳定差异。

Falsification

与匹配的随机价格水平无统计差异。

OOS Acceptance

至少部分 level category 对：

- * touch probability;
- * reaction;
- * continuation;
- * target function;

存在稳定信息。

——

8.4 Liquidity Sweep

Domain Meaning

价格短暂穿越 candidate liquidity level 后重新返回原区域。

Operational Definition

例如：

1. 至少穿越水平 δ ;

2. 在 N 根 bar 内重新收盘到水平另一侧；

3. 记录：

- * penetration；
- * recovery speed；
- * volume；
- * duration。

必须区分

touch

penetration

sweep / rejection

acceptance

禁止：

只要 wick 穿过 swing 就全部叫 sweep。

Hypothesis

Sweep 后是否更容易出现：

- * order flow reversal；
- * displacement；
- * MSS；
- * opposite continuation。

Falsification

Sweep 与：

- * ordinary touch；
- * ordinary wick；
- * random penetration；

没有显著区别。

OOS Acceptance

在控制：

- * trend;
- * volatility;
- * session;

后仍有 incremental information。

——

8.5 Acceptance

Domain Meaning

价格突破某结构水平以后，在水平外侧持续交易。

Operational Definition

例如：

- * 连续 M 根 close 位于水平外；
- * 或外侧成交时间达到阈值；
- * 或外侧成交 volume 达到预注册阈值。

Hypothesis

Acceptance 是否比普通 boundary crossing 更容易产生同向 continuation。

Falsification

与普通突破无差异。

OOS Acceptance

控制：

- * trend;
- * volatility;
- * session;

之后仍具有增量解释力。

Sweep vs Acceptance

它们是竞争结果：

level penetration

|—— returns inside → Sweep

|—— remains outside → Acceptance

——

8.6 Displacement

Domain Meaning

快速、高效率、方向清晰的价格交付。

第一阶段不要直接定义单一阈值

禁止一开始：

range > 1.2 ATR → displacement

优先保存连续特征，例如：

body / ATR

true_range / ATR

close_location_value

directional_efficiency

volume_zscore

same_direction_bar_count

fvg_created

之后形成：

displacement_score

Hypothesis

高 displacement score 是否更容易产生：

- * structure break;
- * continuation;
- * order-flow imbalance。

Falsification

如果：

- * score 与未来结构结果无关；
- * 只在单一市场时期有效；
- * MBO 中没有对应 microstructure 特征；

则推翻当前 displacement 定义。

OOS Acceptance

要求：

- * 与结构结果存在近似单调关系；
- * 跨时期稳定；
- * 最终最好可以与 MBO 证据对应。

8.7 Raw Boundary Break 与 BOS

不要把 break detector 一开始就压缩成 BOS。

保留两个事件：

raw_boundary_break
qualified_bos

Raw Boundary Break

定义：

close > structural boundary + break_buffer

或 bearish 对称规则。

BOS

如果 raw break:

- * 与当前 structure direction 一致;
- * 满足 BOS qualification;

才派生:

qualified_bos

Hypothesis

BOS 是否比普通 boundary break:

- * continuation 更高;
- * MFE 更大;
- * protected swing survival 更强。

Displacement 是否提高 BOS 信息价值。

Falsification

如果 BOS 与匹配的 ordinary break 无差异:

- * BOS 定义无价值;
- * 或 structure direction 本身无额外信息。

8.8 Protected Swing

Domain Meaning

导致当前结构突破, 并维持当前 structure thesis 有效的反向 swing。

Operational Definition

最近一次 BOS 的 origin leg 所对应的反向 structural swing。

它持续 protected, 直到满足预注册 invalidation。

必须固定 invalidation 类型

不能混用:

wick through

close through

acceptance beyond

需要明确当前 semantic version 使用哪一种, 或者把它们分别保留为不同事件, 再由 higher-level semantic qualification 决定正式 invalidation。

Hypothesis

Protected Swing 是否比同层级普通 swing:

- * reaction 更强;

- * survival 更高;

- * 对结构失效更有意义。

Falsification

Protected swing 与同等级 ordinary swing 无显著差异。

8.9 MSS Core

Domain Meaning

当前 internal structure 第一次发生反方向 structural boundary break。

Operational Definition

与 current internal direction 相反的第一个:

confirmed structural boundary break

重要原则

MSS Core 中不要硬编码：

- * Sweep;
- * Displacement;
- * FVG;
- * HTF location。

否则无法进行 ablation。

基础事件：

event:

type: mss_core

上下文：

context_tags:

prior_sweep: true

displacement_score: 0.81

fvg_created: true

parent_location: discount

Brain 经过研究以后可以使用：

qualified_mss_setup

但 Trading Eye 的底层语义必须保持原子化。

需要独立回答

- * MSS 本身有没有价值？
- * Sweep 增加多少？
- * Displacement 增加多少？
- * FVG 增加多少？
- * HTF context 增加多少？
- * MBO 增加多少？

8.10 FVG

Domain Meaning

三个连续 bar 之间形成的 price non-overlap region。

Operational Definition

Bullish:

$\text{Low}[t] > \text{High}[t-2] + \text{min_gap}$

Bearish 对称。

在第 3 根 bar close 时确认。

Hypothesis

研究:

- * FVG size;
- * source displacement;
- * first retracement depth;

与未来 continuation 的关系。

Falsification

FVG 与匹配的 random zone 无差异。

OOS Acceptance

控制:

- * trend;
- * volatility;
- * session;
- * distance;

之后特定 FVG category 仍有稳定价值。

生命周期

每个 FVG 必须维护：

open

partially_filled

midpoint_touched

fully_filled

invalidated

expired

生命周期变化通过事件完成，不修改历史事件。

——

8.11 Order Block / Origin Zone

Domain Meaning

导致 qualifying structural impulse 的 origin price region。

Operational Definition

预注册为：

* qualifying impulse 前最后一根 opposite candle；

* 或连续 opposite candle cluster。

区域计算方式必须固定一种，例如：

full_range

body

open_to_low/high

不能根据结果动态挑选。

Hypothesis

首次回测是否优于：

- * ordinary impulse origin;
- * matched random zone。

Outcomes

研究：

- * MAE;
- * MFE;
- * first-touch reaction;
- * fill-to-continuation。

原则

OB 应视为：

derived origin zone

而不是假设存在神秘机构订单的独立原因。

8.12 Active Dealing Range

Domain Meaning

当前结构正在进行价格交付的有效外部区间。

Operational Definition

由最近有效的相反 structural swings 构成。

必须记录：

origin_event_ids
range_created_at
known_at

直到出现有效 external acceptance break 才更新。

禁止

每根 bar 根据 hindsight 重新选择最有利的 range。

生命周期

created

active

extended

invalidated

replaced

Hypothesis

Normalized range location 是否影响：

- * setup outcome;
- * target probability;
- * retracement;
- * continuation。

——

8.13 Premium / Discount / IRL / ERL

Operational Definition

保存连续变量：

$$x = (\text{price} - \text{range_low}) / (\text{range_high} - \text{range_low})$$

例如：

$$\text{normalized_location} = 0.27$$

而不仅是：

discount = true

ERL:

- * active range boundary;
- * boundary 外 candidate liquidity pools。

IRL:

- * internal structure levels;
- * internal zones。

Hypothesis

Premium / Discount 是否只有在:

- * 特定 direction;
- * delivery phase;
- * parent relationship;

下才存在意义。

Falsification

如果所谓 discount long advantage:

- * 与其他位置没有区别;
- * 或只是 trend 的替代变量;

则不能作为独立 signal。

——

8.14 Delivery Phase

Domain Meaning

当前价格位于:

balance

expansion
retracement
reversal_attempt
transition

中的哪一种 delivery phase。

Operational Inputs

可使用：

- * active leg;
- * structure direction;
- * protected swing;
- * range extension;
- * volatility compression。

Hypothesis

同一个：

- * Sweep;
- * MSS;
- * FVG;

在不同 phase 是否有不同结果。

重要的 MTF 解释

例如：

child bearish
+
parent bullish
+
parent protected swing intact

并不是“时间框架冲突”。

更合理的是：

child bearish retracement inside parent bullish delivery

8.15 DOL Candidate 与 DOL Hypothesis

这是 Eye 与 Brain 最重要的职责边界之一。

Trading Eye

只输出 candidate targets, 例如:

3 BSL above

2 SSL below

以及每一个候选:

- * price;
- * distance;
- * hierarchy;
- * type;
- * age;
- * timeframe;
- * session relevance;
- * current state。

Eye 只能定义:

dol_candidate

Trading Brain

Brain 对这些 candidate:

- * ranking;
- * probability;
- * expiry;
- * path dependency;

最终产生：

dol_hypothesis

例如：

Most likely DOL:

1H BSL 21560

calibrated probability:

0.58

Hypothesis

DOL ranking 是否优于：

- * nearest liquidity;
- * simple trend direction;
- * random target。

——

九、Trading Eye 内部组织

9.1 Data Normalizer

负责：

- * contract normalization;
- * trading session normalization;
- * OHLCV construction;
- * timeframe aggregation;
- * MBO event ordering;
- * missing data QA;
- * timestamp normalization;
- * rolling futures contract handling。

Data Normalizer:

不产生任何 SMC interpretation。

9.2 Semantic Event Engine

负责从标准化数据产生原子 semantic events, 例如:

swing_confirmed
liquidity_level_created
level_touched
level_penetrated
sweep_confirmed
acceptance_confirmed
fvg_created
fvg_touched
raw_boundary_break
bos_confirmed
mss_core_confirmed
displacement_observed

所有事件:

immutable

如果状态改变:

创建新事件, 而不是修改过去事件。

9.3 Timeframe State Reducer

所有 timeframe 使用同样的 reducer 设计:

old state + new event → new state

例如:

5m_state_t
+
5m_bos_event

=
5m_state_t+1

必须保证:

same raw data + same semantic version = same event stream + same final state

即完全 deterministic。

9.4 Cross-Timeframe Relation Resolver

这个模块可以解释确定性的结构关系，但不能进行概率交易判断。

例如:

parent_tf: 1h
child_tf: 5m
parent_direction: bullish
parent_protected_low_intact: true
child_internal_direction: bearish
relationship:
 role: parent_retracement
 parent_state_invalidated: false
 reversal_warning: false

推荐关系:

aligned_expansion
parent_retracement
reversal_attempt
parent_transition
balance_inside_parent
unresolved

9.5 Market Snapshot Publisher

Trading Eye 对 Brain 提供两类输出：

Event Stream

回答：

What just happened?

Current Snapshot

回答：

What is currently true?

Brain 不应该重新扫描 OHLCV 来推导同一批结构事实。

十、多时间框架运行规则

10.1 父周期只能由自身确认事件正式改变

例如：

5m bearish

不能直接将：

1H bullish

改成 bearish。

5m 可以产生：

parent_reversal_warning = true

但 1H 的正式结构只能通过：

1H break

1H acceptance

1H invalidation

等预注册规则改变。

10.2 子周期反向结构默认优先解释为父周期回撤

例如：

1H bullish
1H protected low intact
5m bearish

优先：

5m bearish retracement inside 1H bullish state

而不是：

1H bullish vote
5m bearish vote
=> neutral

原则：

Timeframes are not a democracy.

10.3 子周期可以提供 reversal evidence

例如：

5m BSL sweep
↓
bearish displacement
↓
5m MSS
↓

15m protected low break

可以逐步增加：

1H reversal hypothesis

但 Trading Eye 不能提前改写：

1H structure fact

Brain 可以降低 bullish continuation belief。

Eye 只能更新确定性事实。

10.4 Parent = constraint, Child = path/timing

推荐角色：

Context:

4H

external range

major liquidity

protected swing

candidate target

Setup:

1H

delivery phase

retracement zone

active POI

Trigger:

15m / 5m

sweep

displacement

MSS

reclaim

Execution:

1m

entry price

limit mechanics
stop
fill management

10.5 取消 4H 第一层 hard AND gate

当前研究目标不应该使用：

4H not fully bullish
→ reject every LTF long setup

4H 应提供：

context
target map
range
location
invalidation

低周期事件必须保留，并增加标签：

aligned_with_parent
against_parent_but_parent_intact
parent_neutral
parent_reversal_candidate

之后通过研究确定它们是否具有不同 expectancy。

十一、独立 Cross-Timeframe Relation Object

不要把 parent state 所有字段复制到 child state。

维护独立 relation object，例如：

relation_id: 1h__5m
parent_tf: 1h

```
child_tf: 5m
parent:
  structure_direction: bullish
  phase: expansion
  protected_low_intact: true
child:
  internal_direction: bearish
  phase: retracement
relationship:
  role: parent_retracement
  child_location_in_parent_range: 0.34
  parent_invalidation_distance_atr: 1.72
  aligned_target_candidates:
    - 1h_bsl_21560
    - previous_day_high_21578
warnings:
  child_mss_against_parent: false
  parent_protected_swing_under_attack: false
```

十二、Session 是独立上下文对象

不要把 Session 塞进某一个 timeframe。

Session 横跨全部 timeframe。

例如：

```
session:
  name: new_york_am
  phase: opening_expansion
  session_open:
  opening_range_high:
  opening_range_low:
  prior_day_high:
  prior_day_low:
  overnight_high:
  overnight_low:
  elapsed_minutes:
```

realized_volatility:
relative_volume:

每个 semantic event 都应该能够附带：

session_name
session_phase
minutes_from_session_open

例如区分：

- * pre-open;
- * first 15 minutes;
- * opening expansion;
- * mid-session;
- * close。

这样才能研究：

同样的 Sweep / MSS / Displacement 在不同 session phase 是否具有不同价值。

Session 第一阶段：

research feature

而不是 hard gate。

——

十三、Trading Brain：维护多个竞争性市场假设

不要使用：

current_belief = bullish

Brain 应维护多个并行 hypotheses。

例如：

hypotheses:

- id: H1
 - path: continuation_to_1h_bsl
 - target: 21560
 - support:
 - 1h_protected_low_intact
 - 5m_ssl_sweep
 - 5m_bullish_displacement
 - contradiction:
 - 5m_fvg_failed
 - invalidation: 21482.25
 - probability: 0.58
 - expiry: end_of_ny_am
- id: H2
 - path: deeper_1h_retracement
 - target: 21470
 - support:
 - weak_5m_follow_through
 - 15m_bearish_internal_structure
 - invalidation: 21541.75
 - probability: 0.29
- id: H3
 - path: full_1h_reversal
 - target: 21420
 - support: []
 - invalidation: 21541.75
 - probability: 0.13

每一个新 event:

改变 hypothesis 的相对可信度，而不是直接翻转整个市场方向。

——

十四、交易惯性的正式表达

“交易惯性”不要理解为：

market must remain bullish until one switch event

更准确的是：

Market beliefs are persistent but continuously updated by evidence.

可以使用类似：

$$\begin{aligned} \text{LogOdds_new} &= \\ &\text{LogOdds_old} \\ &+ \\ &\text{EvidenceWeight} \\ &- \\ &\text{TimeDecay} \end{aligned}$$

但第一版不需要直接建设复杂 Bayesian system。

可以首先使用：

calibratable scoring model

要求：

- * 每次 belief update 可解释；
- * 记录 supporting evidence；
- * 记录 contradictory evidence；
- * 记录 invalidation；
- * 可 replay；
- * 可 calibration。

——

十五、Signal Research 与 Execution Research 必须分离

这是整个系统的重要研究边界。

15.1 Signal Research

回答：

Given the current market state, what is more likely to happen next?

研究：

target_before_invalidation probability
future MFE
future MAE
next structural event
continuation distance
time to target
signal half-life

Signal Research 不先回答：

FVG50 还是 market entry 更赚钱？

15.2 Execution Research

回答：

Given that a statistically useful signal exists, how should it be converted into a real order?

比较：

market entry
FVG50 limit
OB50 limit
reclaim entry
breakout entry
passive limit
aggressive limit

以及：

fill probability
missed opportunity
adverse selection
slippage
stop distance
time stop

cancel logic

不能把：

没有成交

误判成：

signal 本身无效。

——

十六、事件研究：Nested Event Chain

建立事件链：

C0 = matched control event

E1 = Candidate level touch

E2 = Sweep

E3 = Sweep + Displacement

E4 = Sweep + Displacement + MSS

E5 = Sweep + Displacement + MSS + FVG

E6 = + Parent/Child Context

E7 = + MBO Confirmation

每一步：

$$\Delta_i = \text{Metric}(E_i) - \text{Metric}(E_{i-1})$$

例如：

Sweep	53%
Sweep + Displacement	61%
Sweep + Displacement + MSS	64%
+ FVG	63%
+ HTF Context	69%
+ MBO Confirmation	74%

这种结果可以证明：

- * Displacement 有明显增量；
- * MSS 有少量增量；
- * FVG 未增加价值；
- * HTF context 有价值；
- * MBO confirmation 有较大价值。

不要假设 SMC 理论中的每个组件都必须有效。

十七、Non-Nested Comparison

Nested chain 可能掩盖 interaction。

因此还必须比较：

MSS without Sweep

vs

MSS with Sweep

Displacement without MSS

vs

Displacement with MSS

FVG without Displacement

vs

FVG with Displacement

LTF setup aligned with HTF

vs

LTF setup against HTF but parent intact

vs

LTF setup after parent invalidation

后续可以使用：

- * stratified regression；
- * survival analysis；
- * conditional probability tables；
- * tree models；
- * calibrated probability models。

十八、Matched Controls

禁止将某个 SMC event 与“全部普通 bar”直接比较。

例如研究 Sweep, control 至少要尽可能匹配：

- same instrument
- same session phase
- similar volatility
- similar trend regime
- similar distance to level
- same timeframe
- similar volume environment

否则可能得到错误结论：

Sweep works

实际上只是在测：

NY open has higher volatility

建议支持：

- matched control events
- pseudo-level controls
- time-shifted events
- same-session non-sweep touches

十九、Structural Outcomes 必须先于 P&L

Signal Research 第一阶段优先研究结构结果，而不是最终收益。

至少维护以下 outcome。

Direction / Structure

target_before_invalidatation
next_structural_event
next_bos_direction
protected_swing_survival

Price Path

MFE / ATR
MAE / ATR
MFE / MAE
continuation_distance
retracement_depth
range_extension

Time

time_to_target
time_to_invalidatation
time_to_first_retest
signal_half_life

Zone

FVG first-touch probability
FVG50 touch probability
OB touch probability
reaction_after_touch
failure_after_touch

只有这些 structural outcomes 足够稳定后，才进一步测试：

1R
2R
3R
stop methods
entry methods
fees
slippage

二十、OHLCV 与 MBO 的研究边界

OHLCV 验证：

Price Geometry Layer

问题：

某个价格结构标签是否与未来市场路径存在稳定统计关系？

MBO 验证：

Microstructure Mechanism Layer

问题：

OHLCV semantic 背后是否存在一致、可观察的 order-flow behavior？

二者不能混为一层。

——

二十一、OHLCV — MBO 对照关系

Sweep

OHLCV：

penetrate liquidity level
then recover

MBO：

研究是否出现：

aggressive order increase during penetration
absorption
replenishment
OFI reversal

Acceptance

OHLCV:

cross level
remain outside

MBO:

persistent same-direction order flow
queue migration
lack of meaningful replenishment

Displacement

OHLCV:

large directional candle
high efficiency
fast movement

MBO:

higher trade velocity
directional OFI
queue depletion
higher price impact per unit volume

MSS

OHLCV:

opposite structural boundary break

MBO:

研究结构 break 前后:

order-flow sign

depth relationship

queue behavior

是否发生系统性变化。

FVG Retest

OHLCV:

return into imbalance zone

then react

MBO:

absorption

replenishment

re-acceleration

Failed FVG

OHLCV:

retracement passes through zone

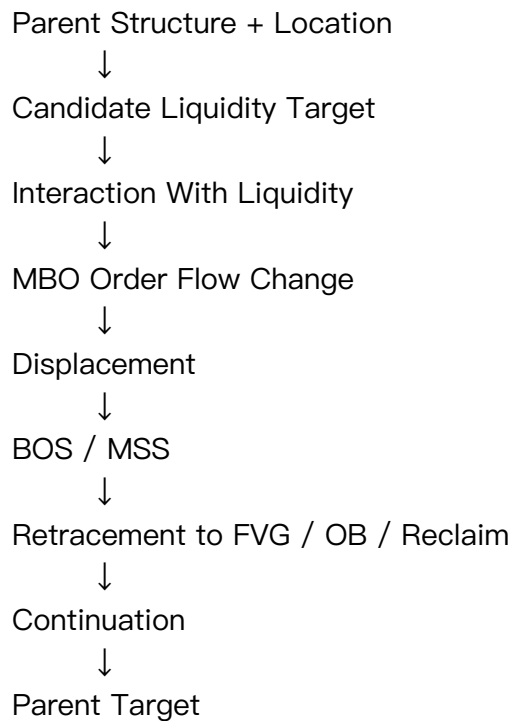
MBO:

lack of defensive flow

persistent opposite queue depletion

二十二、完整因果链必须拆成独立假设

整个理论路径可以表达为：



但：

每一个箭头都必须是单独的 empirical hypothesis。

禁止直接把整条链当成事实写进算法。

例如必须分别测试：

Parent structure → candidate target probability
Liquidity interaction → order flow change
Order flow change → displacement
Displacement → structural break
Structural break → FVG/OB retest
Retest → continuation
Continuation → parent target

——

二十三、Experiment Preregistration

预注册不仅针对 semantics。

每一个正式 experiment 应存在 manifest:

experiment_id:
semantic_version:
dataset_version:
instrument_universe:
contract_handling:
session_definition:
train_period:
validation_period:
out_of_sample_period:
event_definition:
control_definition:
parameter_search_space:
primary_outcome:
secondary_outcomes:
primary_metric:
minimum_sample_requirement:
cost_assumptions:
multiple_testing_method:
acceptance_rule:
rejection_rule:

以下内容进入正式测试前必须冻结:

semantic definition
dataset split
primary metric
control group
parameter search space
success criteria

每次修改必须:

- * 升级 semantic version;
- * 或创建新的 experiment version。

二十四、Trading Brain 推荐结构

24.1 Hypothesis Manager

至少能够表示：

continuation
deeper_retracement
reversal
balance
failed_breakout

每一个 hypothesis 必须有：

id
path
target
support
contradictions
invalidation
score/probability
expiry
status

24.2 Belief Updater

根据 Trading Eye 新事件：

increase
decrease
invalidate
expire

hypotheses。

必须保存：

- * 哪个 event 增加 belief;
- * 哪个 event 减少 belief;
- * magnitude;
- * old score;
- * new score;
- * update timestamp;
- * semantic version / model version。

24.3 Signal Policy

根据 Signal Research 结果回答：

Is there a tradable edge?
What structural outcome is expected?
How long is the signal valid?
What invalidates the signal?
What evidence is still missing?

24.4 Trade Intent Builder

Brain 不直接发送订单。

输出：

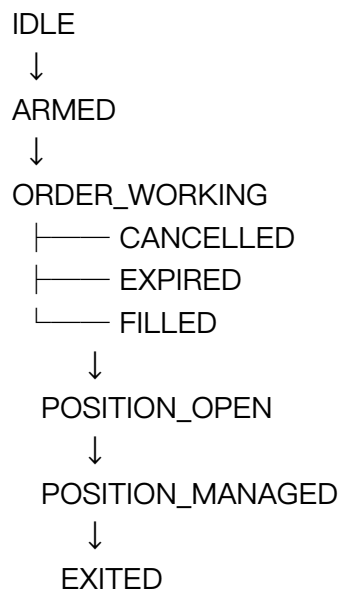
side:
signal_id:
preferred_entry_methods:
invalidation:
structural_targets:
max_wait:
risk_budget:
cancel_conditions:

然后交给 Execution。

二十五、Execution 必须拥有独立订单状态机

市场状态机与订单状态机严格分离。

例如：



订单状态机负责解决：

- * FVG50 未成交怎么办；
- * 什么时候切换 reclaim；
- * 什么时候允许 market；
- * 限价等待多久；
- * partial fill 怎么办；
- * stop 什么时候移动；
- * target 什么时候缩短；
- * signal invalidated 后如何 cancel；
- * stale order 怎么处理。

不要把这些行为塞进 Trading Brain。

——

二十六、推荐项目目录

目标结构可以参考：

```
smc_test/  
  data/  
    ingestion/  
    normalization/  
    bar_builder/  
    mbo_reconstruction/  
  semantics/  
    registry.yaml  
    parameters.yaml  
    swing_detector.py  
    liquidity_detector.py  
    displacement_detector.py  
    structure_detector.py  
    imbalance_detector.py  
  events/  
    schemas.py  
    event_store.py  
    replay.py  
  eye/  
    timeframe_state.py  
    timeframe_reducer.py  
    relation_state.py  
    relation_resolver.py  
    session_state.py  
    market_snapshot.py  
  research/  
    semantic_qa/  
    event_studies/  
    matched_controls/  
    structural_outcomes/  
    mtf_relationships/  
    mbo_validation/  
    ablation/  
    walk_forward/  
  brain/  
    hypotheses.py  
    belief_update.py  
    dol_ranking.py
```

- signal_model.py
- trade_intent.py
- execution/
 - entry_models.py
 - fill_models.py
 - order_state_machine.py
 - risk_manager.py
 - position_manager.py
- experiments/
 - manifests/
 - results/
 - reports/
- tests/
 - unit/
 - replay/
 - leakage/
 - determinism/

但是：

该目录只是目标职责结构，不是要求机械搬迁所有现有代码。

如果当前项目已经存在合理目录：

优先渐进迁移。

二十七、实施顺序

不要一次同时重写全部 SMC 系统。

严格按照 dependency 顺序推进。

Phase 0 — Existing System Audit

首先：

- * 扫描 repository;
- * 识别已有模块;
- * 映射现有数据流;
- * 找未来数据泄漏风险;
- * 找 state coupling;
- * 找重复 semantic logic;
- * 找现有 research / execution coupling;
- * 找可以直接复用的代码。

输出:

Current Architecture
Reusable Components
Conflicting Components
Missing Components
Refactor Plan

之后再改代码。

Phase 1 — Auditable Foundation

首先实现或确认:

event_time
known_at
semantic_version
immutable event store
deterministic replay

并建立:

leakage tests
determinism tests
replay tests

这是后续所有工作的基础。

Phase 2 — Core Atomic Semantics

优先实现：

Confirmed Swing
Structural Leg
Candidate Liquidity Level
Sweep
Acceptance
Raw Boundary Break
FVG
Displacement Features

这些必须尽可能原子化。

Phase 3 — Derived Structure

然后建立：

Qualified BOS
Protected Swing
MSS Core
Active Dealing Range
Delivery Phase

这些必须由底层 immutable events 派生。

Phase 4 — Market State Architecture

建立：

TimeframeState
TimeframeReducer
RelationState
RelationResolver

SessionState
MarketSnapshot

验证：

same data
+
same semantics
=
same state

Phase 5 — Signal Research

先研究：

single events
event combinations
HTF / LTF relationships
session relationships
structural outcomes
matched controls
ablation

不要先调 entry。

Phase 6 — MBO Validation

加入 MBO，并验证：

Sweep
Acceptance
Displacement
MSS
FVG Retest

是否有对应稳定 microstructure evidence。

Phase 7 — Trading Brain

当 Eye semantics 足够稳定以后再建立：

Hypothesis Manager

Belief Updater

DOL Ranking

Signal Policy

Trade Intent

第一版优先：

interpretable + calibratable

不要立即建设复杂 neural/Bayesian architecture。

Phase 8 — Execution Research

Signal edge 基本确认以后，再比较：

FVG50

OB50

reclaim

market

different wait times

different cancel rules

different stops

different targets

研究：

fill probability

miss rate

adverse selection

slippage

realized expectancy

Phase 9 — Shadow Live

最终进入：

real-time Eye

+

real-time Brain

+

no real orders

检查：

- * state 是否稳定；
- * event 是否重复；
- * known_at 是否正确；
- * parent/child relationship 是否频繁无意义跳变；
- * belief 是否根据 evidence 合理变化；
- * realtime 与 historical replay 是否一致；
- * 同一段数据 realtime 与 replay 是否生成一致 semantic stream；
- * signal expiry 是否合理；
- * DOL ranking 是否稳定。

Shadow Live 通过后，再考虑真实 execution。

二十八、研究阶段不要过早使用 hard gate

当前系统曾经使用类似：

bias

→ range

→ poi

→ displacement

→ volume

→ target

→ execution

的 hard AND pipeline。

新的研究架构应优先：

retain event

+

attach context

+

measure outcome

而不是：

one condition failed

→ delete observation

特别是：

4H bias

session

FVG

MSS

MBO

这些第一阶段更应该是：

features / context tags

然后通过 incremental research 判断是否应该成为真正 gate。

最终 production policy 可以使用 gate。

Research pipeline 不应该提前把不知道是否有价值的信息硬编码为 gate。

二十九、必须支持原子语义与复合 Setup 分离

Trading Eye 可以输出：

Sweep
Displacement
MSS
FVG
HTF relation
Session phase

这些必须独立存在。

经过研究后 Brain 才可以形成：

Qualified Long Continuation Setup

例如：

Sweep
+
high displacement score
+
MSS core
+
parent retracement
+
NY opening phase

但不能反向修改语义定义，使 MSS 自动意味着整个组合。

原则：

Semantic layer preserves observations.

Brain layer constructs hypotheses and setups.

三十、系统必须支持 Replay 与 Audit

对于任意 signal 或 trade，应能够追溯：

raw data
↓
atomic events
↓
derived events
↓
timeframe states
↓
cross-timeframe relations
↓
market snapshot
↓
hypothesis state
↓
belief updates
↓
signal
↓
trade intent
↓
execution decisions

需要能够回答：

Why did this trade exist?
What did the system know at that exact time?
Which events caused the belief change?
Which semantic version produced those events?
Would replay produce the same answer?

——

三十一、关键工程不变量

以下应视为 architectural invariants。

Invariant 1

No semantic can be consumed before known_at.

Invariant 2

Historical semantic events are immutable.

Invariant 3

Same input + same semantic version = same output.

Invariant 4

Child timeframe cannot directly mutate parent timeframe structure.

Invariant 5

Trading Eye does not output trade decisions.

Invariant 6

Trading Brain does not re-detect raw market structure.

Invariant 7

Execution does not reinterpret SMC structure.

Invariant 8

Research semantics and production semantics share the same definitions.

Invariant 9

Signal quality and execution quality are measured separately.

Invariant 10

No OOS information may be used to select semantic parameters.

三十二、测试要求

必须逐步补齐以下测试。

Semantic Unit Tests

覆盖：

Swing

Sweep

Acceptance

FVG

Raw Break

BOS

MSS

Protected Swing

Range lifecycle

包括 boundary conditions。

Known-at / Leakage Tests

必须构造案例验证：

$\text{event_time} < \text{known_at}$

时，系统无法在 known_at 前读取事件。

Replay Tests

同一数据 replay 两次：

event stream identical

state stream identical

snapshot identical

Determinism Tests

不同运行：

same raw data
same parameters
same version

必须得到一致 hash / canonical output。

Parent / Child Tests

验证：

5m state change

不能直接修改：

1H formal structure

Lifecycle Tests

例如：

FVG:

open

→ midpoint_touched

→ fully_filled

以及：

Range:

created

→ active

→ invalidated

→ replaced

必须严格可 replay。

三十三、不要为了重构而重构

以下行为应避免：

- * 创建大量 abstraction 但没有实际使用；
- * 为未来可能需求构建复杂 distributed architecture；
- * 提前创建 event bus / microservices；
- * 为简单 reducer 引入大型 framework；
- * 一次移动整个项目目录；
- * 重写所有已经正确工作的 indicator；
- * 一开始引入 neural network；
- * 一开始构建复杂 Bayesian inference；
- * 同时调整 semantics、research、brain 和 execution，导致无法 attribution。

每一个 phase 应尽可能：

small
testable
replayable
auditable
reversible

三十四、当前重构最重要的目标不是盈利最大化

这一轮重构第一目标不是：

maximize backtest P&L

而是建立一个能够可信回答以下问题的系统：

What exactly happened?
When did the system know it?
What did that event change?
Did the semantic have statistical value?

How much incremental information did it add?
Was the result stable out-of-sample?
Was poor performance caused by signal or execution?
Can the exact decision be reproduced?

只有这些问题可以可靠回答后，才进一步优化盈利。

——

三十五、最终系统职责边界

Trading Eye

Observe market facts
Detect preregistered semantics
Maintain timeframe states
Maintain session context
Resolve parent-child relationships
Publish events
Publish market snapshots

Trading Brain

Generate competing path hypotheses
Maintain and update beliefs
Rank DOL candidates
Evaluate signal edge
Determine whether to trade
Build Trade Intent

Execution

Choose execution method
Manage limit / market orders
Cancel / replace
Manage fills
Manage stop loss
Manage take profit
Manage position

这三个层级必须保持清晰依赖方向：

Data
↓
Semantics
↓
Trading Eye
↓
Trading Brain
↓
Trade Intent
↓
Execution

禁止产生反向耦合，例如：

execution result
→ retroactively changes historical semantic event

——

三十六、Codex 实际执行方式

不要仅给我架构建议。

你需要：

1. 检查当前代码库。
2. 对照本规范建立现状矩阵。
3. 判断哪些已经存在。
4. 判断哪些实现错误或职责混乱。
5. 制定最小侵入式迁移方案。
6. 按上述 Phase 顺序开始实施。
7. 修改代码时同步补测试。
8. 不要为了“目录完全符合目标”而做无意义迁移。
9. 每一阶段完成后验证：
 - * behavior;
 - * leakage;
 - * replay;
 - * determinism;

* backward compatibility。

10. 如果发现本规范与现有系统某处存在合理冲突：

- * 不要机械执行；
- * 解释冲突；
- * 判断哪个设计更满足总体目标；
- * 选择最小且正确的实现。

——

三十七、每一阶段的输出要求

每完成一个阶段，输出：

A. Existing Behavior

修改前系统如何工作。

B. Problem

为什么需要修改。

C. Changes Made

具体修改了哪些文件和逻辑。

D. Design Decisions

为什么采用当前方案。

E. Tests

新增和执行哪些测试。

F. Validation

测试和 replay 是否通过。

G. Remaining Gaps

还有哪些未完成。

H. Next Phase

下一阶段准备实施什么。

不要仅输出：

Implemented successfully.

——

三十八、宣布任务完成前必须进行最终自审

在声称整个重构或某一个阶段完成前，必须自行审查代码。

重点检查：

Architecture

- * Eye / Brain / Execution 是否真正分离？
- * 有没有 Brain 重新检测 structure？
- * 有没有 Eye 在做 probability/trading decision？
- * 有没有 Execution 重新判断 market structure？

Time / Leakage

- * 所有需要右侧确认的语义是否正确设置 known_at？
- * replay 是否可能提前看到 future event？
- * parent state 是否可能被 child hindsight 修改？

Semantics

- * 语义是否原子化？
- * MSS 是否错误包含 Sweep / FVG / Displacement？
- * BOS 是否与 raw break 分离？
- * Sweep 是否与 penetration / acceptance 分离？
- * candidate liquidity 是否被错误宣称成真实订单？

State

- * events 是否 immutable?
- * reducer 是否 deterministic?
- * lifecycle 是否通过事件而不是直接改历史?
- * relation state 是否与 timeframe state 分离?

Research

- * 是否存在 matched controls?
- * 是否先看 structural outcomes?
- * 是否混淆 signal failure 与 fill failure?
- * 是否存在 hard gate 导致研究样本被提前删除?
- * train / validation / OOS 是否真正隔离?

Execution

- * market state machine 与 order state machine 是否分离?
- * limit / reclaim / market 是否可以独立比较?
- * cancel / expiry / partial fill 是否有清楚规则?

Engineering

- * 是否重新实现了已有成熟功能?
- * 是否存在不必要 abstraction?
- * 是否过度建设 infrastructure?
- * 是否有 dead code?
- * 是否存在 duplicate semantic definitions?
- * 是否有测试缺口?

如果发现问题:

在宣布完成前直接修复。

三十九、最终目标

最终系统不应该只是一个“把 SMC 规则编码出来的交易机器人”。

它应该成为:

一个 deterministic、event-sourced、multi-timeframe、preregistered、falsifiable、research-driven market reasoning system。

整个开发过程遵循：

Observe
↓
Define
↓
Replay
↓
Measure
↓
Falsify
↓
Calibrate
↓
Reason
↓
Trade
↓
Execute

而不是：

SMC theory says X
↓
encode X
↓
optimize backtest P&L

最重要的最终边界仍然是：

Trading Eye

看见事实，维护确定性市场状态。

Trading Brain

对未来提出竞争性、可证伪的假设并更新信念。

Execution

把已经确认具有统计优势的 Trade Intent 转化为真实订单并管理风险。

四十、开始任务

现在开始时不要立即大规模修改代码。

首先完成：

Repository Audit

+

Current Architecture Map

+

Existing Capability Matrix

+

Gap Analysis

+

Proposed Minimal Refactor Plan

然后根据依赖关系判断：

Phase 1 当前已有多少？

优先从最基础、影响最大的缺口开始实施。

不要向我询问可以通过代码库自行确定的信息。

如果部分设计细节无法从代码库确定，选择：

最保守、最可审计、最容易以后修改的实现。

最终目标是让整个 smc_test 朝上述目标架构实际运行，而不是仅仅生成新的设计文档。